

Support Vector Machine - Hard Margin and Ensemble Method

Soft Margin SVM

How to adapt the SVM when data has outliers?

$$\begin{aligned} \min_w & \frac{1}{2} \|w\|^2 \\ \text{s. t. } & (w^T x_i) y_i \geq 1 \quad \forall i \end{aligned}$$

- Make every w feasible
- Fix any w , w classifies some points correctly and misclassifies some points
- The incorrectly classified points "pay bribe" to go to the correct side !

Modified Formulation

$$\begin{aligned} \min_{w, \epsilon} & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i \\ \text{s. t. } & (w^T x_i) y_i + \epsilon_i \geq 1 \quad \forall i \\ & \epsilon_i \geq 0 \quad \forall i \end{aligned}$$

Objective Function

$$L(w, \epsilon, \alpha, \beta) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i + \sum_{i=1}^n \alpha_i (1 - w^T x_i y_i - \epsilon_i) - \sum_{i=1}^n \beta_i \epsilon_i$$

Primal and Dual

$$\begin{aligned} \min_{w, \epsilon} \left[\max_{\alpha \geq 0, \beta \geq 0} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i + \sum_{i=1}^n \alpha_i (1 - w^T x_i y_i - \epsilon_i) - \sum_{i=1}^n \beta_i \epsilon_i \right] \\ \equiv \\ \max_{\alpha \geq 0, \beta \geq 0} \left[\min_{w, \epsilon} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \epsilon_i + \sum_{i=1}^n \alpha_i (1 - w^T x_i y_i - \epsilon_i) - \sum_{i=1}^n \beta_i \epsilon_i \right] \end{aligned}$$

Solving the unconstrained optimization problem within the dual, we get

$$w^* = \sum_{i=1}^n \alpha_i x_i y_i$$

and

$$\alpha_i + \beta_i = C, 1 \leq i \leq n$$

By substituting $w_{\alpha, \beta}^*$ into the lagrangian

$$\begin{aligned} \max_{\alpha \geq 0} \quad & \alpha^T 1 - \frac{1}{2} \alpha^T Y^T X^T X Y \alpha \\ & \beta \geq 0 \\ & \alpha + \beta = C \end{aligned}$$

$$\max_{0 \leq \alpha \leq C} \alpha^T 1 - \frac{1}{2} \alpha^T Y^T X^T X Y \alpha$$

If $C \Rightarrow 0$, $\Rightarrow \alpha^* = 0 \in R^n \Rightarrow w^* = 0$

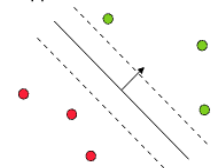
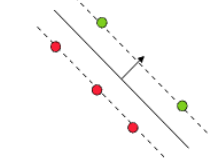
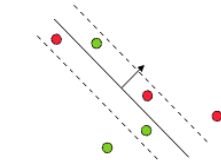
If $C \Rightarrow \infty$, $\Rightarrow$ Hard Margin

- Let (w^*, ϵ^*) be the primal optimal solution
- Let (α^*, β^*) be the dual optimal solution

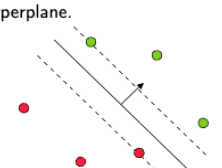
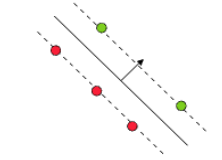
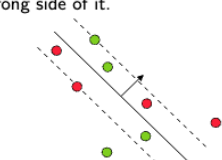
Complimentary Slackness Conditions

- $\forall i \alpha_i^* (1 - w^{*T} x_i y_i - \epsilon_i) = 0$
- $\forall i \beta_i^* (\epsilon_i) = 0$

Primal View

Situation	Inference	Interpretation
$(\mathbf{w}^{*T} \mathbf{x}_i) y_i > 1$	$\xi_i^* = 0$ $\alpha_i^* = 0$	Points that lie on the right side of the correct supporting hyperplane are not support vectors. 
$(\mathbf{w}^{*T} \mathbf{x}_i) y_i = 1$	$\xi_i^* = 0$ $\alpha_i^* \in [0, C]$	Points that lie on the correct supporting hyperplane <i>could</i> be support vectors. 
$(\mathbf{w}^{*T} \mathbf{x}_i) y_i < 1$	$\xi_i^* > 0$ $\alpha_i^* = C$	Points that lie on the wrong side of the correct supporting hyperplane are some of the most important support vectors. 

Dual View

Situation	Inference	Interpretation
$\alpha_i^* = 0$	$\xi_i^* = 0$ $(\mathbf{w}^{*T} \mathbf{x}_i) y_i \geq 1$	Points that are not support vectors lie on or on the right side of the correct supporting hyperplane. 
$\alpha_i^* \in (0, C)$	$\xi_i^* = 0$ $(\mathbf{w}^{*T} \mathbf{x}_i) y_i = 1$	Points that are support vectors with $\alpha_i^* \neq C$ lie on the correct supporting hyperplanes. 
$\alpha_i^* = C$	$\xi_i^* \geq 0$ $(\mathbf{w}^{*T} \mathbf{x}_i) y_i \leq 1$	Points that are the most important support vectors may either lie on the correct supporting hyperplane or on the wrong side of it. 

Ensemble Classifiers

In machine learning, ensemble methods use multiple learning algorithms to obtain better predictive performance than could be obtained from any of the constituent learning algorithms alone. The Purpose of Meta Classifiers is to convert weak learners into strong learners.

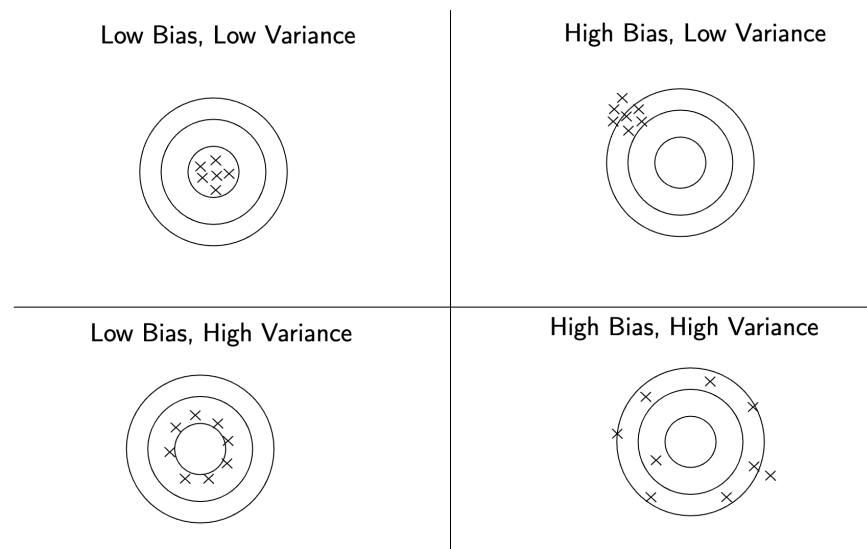
- A weak learner produces a classifier which is only slightly more accurate than random classification. The most commonly used type of weak learning model is the decision tree. This is because the weakness of the tree can be controlled by the depth of the tree during construction. The weakest decision tree consists of a single node that makes a decision on one input variable and outputs a binary prediction, for a binary classification task. This is generally referred to as a “decision stump.”
- A strong classifier is a model for binary classification that performs with arbitrary performance, much better than random guessing. We seek strong classifiers for predictive modeling problems. It is the goal of the modeling project to develop a strong classifier that makes mostly correct predictions with high confidence.

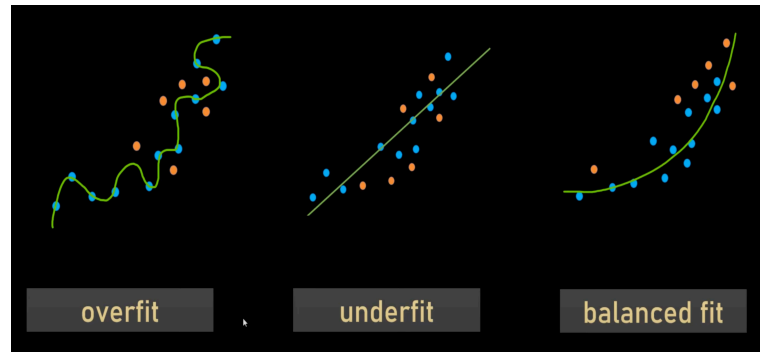
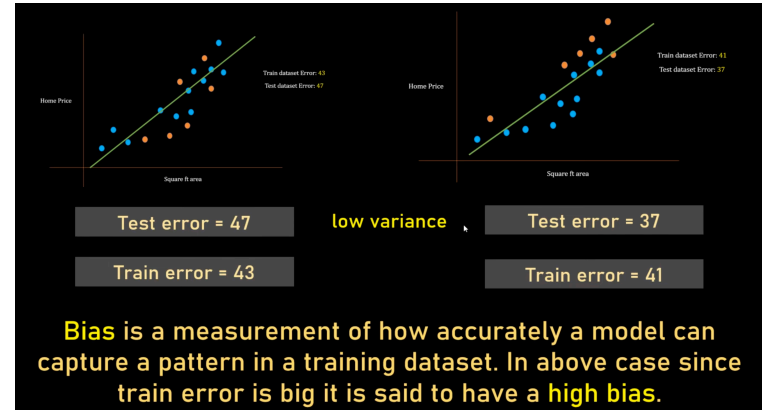
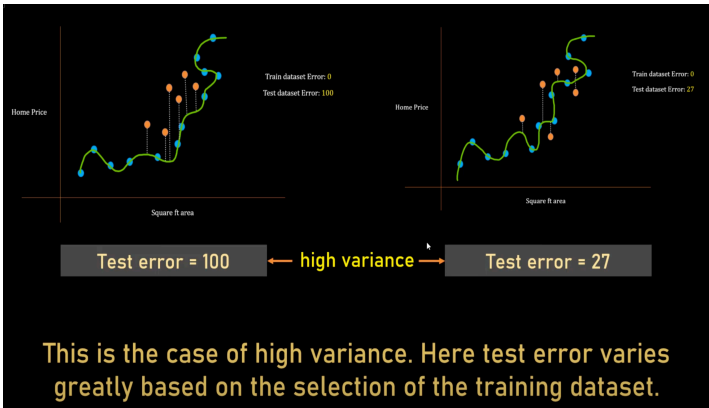
Weak learners are more prone to underfitting rather than overfitting. They are simple models that perform slightly better than random guessing but have high bias and low variance. On the other hand, strong learners are more complex models that can fit the training data well. If not controlled properly, they can overfit the data by capturing noise rather than true patterns, leading to high variance. In ensemble learning, multiple weak learners are combined to form a strong learner, reducing bias while trying to control variance.

Bias and Variance

- **Bias** - This measures the accuracy of the predictions or how close the model is to the true labels. Smaller the error, lower the bias. As the complexity of the model increases, bias tends to decrease. High bias models tend to underfit.
- **Variance** - This measures the variability in the model to fluctuations in the training dataset. As the complexity of the model increases, variance tends to increase. High variance models tend to overfit.

Bias and variance work in opposite directions. Models with a low bias tend to have high variance. Models with a high bias tend to have low variance.





$$\text{Error} = \text{Bias} + \text{Variance}$$

Bias-Variance Trade-off:

- High Bias (Underfitting) → The model is too simple and fails to capture patterns.
- High Variance (Overfitting) → The model is too complex and captures noise along with patterns

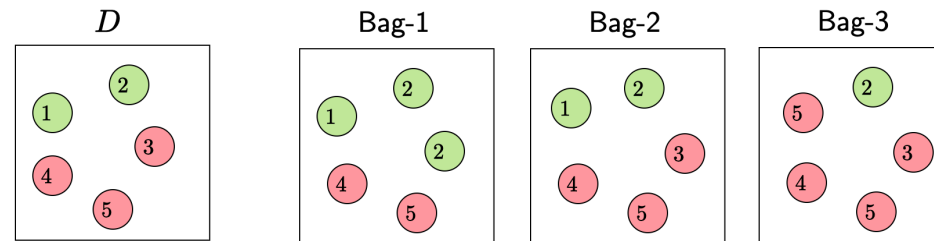
The goal is to find a balance where both bias and variance are minimized for optimal generalization.

Bagging

Bagging or bootstrap aggregation is a technique that tries to reduce the variance. The basic idea is this averaging a set of observations reduces the variance. Instead of averaging observations, we will be averaging over models trained on different datasets. There are two steps:

Bootstrapping

We start with the dataset D and create multiple versions or bags by sampling from D with replacement. Each bag has the same number of data-points as D . Let us call the number of bags B For example with $B = 3$:



Note that points can repeat in a given bag.

Aggregation

We train a model on each bag. Typically, this is a high-variance model, such as a deep decision tree. We call this h_i , the model for the i^{th} bag. If there are B bags, we can aggregate the outputs of all these models.

- For a regression problem, the method of aggregation could be the mean:

$$\frac{1}{B} \sum_{i=1}^B h_i(x)$$

- For a classification problem, the method of aggregation could be the majority vote:

$$\text{sign} \left(\frac{1}{B} \sum_{i=1}^B h_i(x) \right)$$

Distribution of Points of a Bag

Fix a bag and consider an arbitrary data-point, say (x_i, y_i) . The probability that this point gets picked as the first point in the bag is $\frac{1}{n}$. The probability that this point doesn't get picked as the first point is $1 - \frac{1}{n}$. The probability that this point doesn't get picked at all in this bag is $\left(1 - \frac{1}{n}\right)^n$, since we are sampling with replacement (independence). Therefore, the probability that this point appears at least once in a bag is $1 - \left(1 - \frac{1}{n}\right)^n$ (gets picked at all).

$$1 - \left(1 - \frac{1}{n}\right)^n$$

For a very large dataset, this is about $1 - \frac{1}{e} \approx 63\%$. Therefore, about 63% of the data-points in a bag is unique.

Random Forest

Random forests bag deep decision trees with a slight twist. While growing each tree, at each split (question-answer pair), instead of considering all d features, it randomly samples a subset of m features to choose from. m is typically around $\sqrt{d}$. In this way, the resulting trees are uncorrelated. Since the trees are grown independently on their own bags, the algorithm can be run in parallel.

Boosting

A weak learner is a model that does slightly better than a random model. For example, a model that produces an error rate of 0.48 would be a weak model. An error rate of 0.48 would mean that the model produces misclassifies 48% of the data-point. Decision stump is an example of weak model.

In boosting, we combine weak learners, typically high bias models such as decision stumps, and turn them into strong learners, with a low bias. This is achieved by a sequential algorithm that focuses on mistakes committed at each step.

AdaBoost(D , weak-learner)

- $D_0(i) = \frac{1}{n}, 1 \leq i \leq n$
- Repeat for T rounds
 - $h_t \leftarrow$ Train a weak learner on (D, D_{t-1})
 - $\epsilon_t = \text{error}(h_t) = \sum_{h_t(x_i) \neq y_i} D_{t-1}(i)$
 - $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$
 - $\tilde{D}_t(i) = \begin{cases} e^{\alpha_t} \cdot D_{t-1}(i), & h_t(\mathbf{x}_i) \neq y_i, \text{ (mistake)} \\ e^{-\alpha_t} \cdot D_{t-1}(i), & h_t(\mathbf{x}_i) = y_i, \text{ (correct)} \end{cases}$
 - $D_t(i) = \frac{\tilde{D}_t(i)}{\sum_{i=1}^n \tilde{D}_t(i)}$
- Return $\text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t \right)$

Observations:

- Mistakes are boosted by e^{α_t} and correct data-points are brought down by $e^{-\alpha_t}$
- If ϵ_t is high, then e^{α_t} is low. That is, if the classifier in round t has a higher error, it is assigned a lower weight in the final ensemble.

It can be shown that as the number of rounds increases, the training error keeps decreasing. The training error of the final classifier after T rounds is at most:

$$\exp\left(-2 \sum_{i=1}^n \gamma^2 t\right)$$
$$\gamma t = \frac{1}{2} - \epsilon_t$$

Reference

Arun Raj Kumar, Lecture Slides for Machine Learning Techniques
Karthik Thiagarajan, Machine Learning Techniques Github Notes.
Indian Institute of Technology Madras , CS2007 Course Website
Stanford University, CS229 Machine Learning Notes

Thank You !